Learning to Route Under Load: An Inductive Graph Network that Amortizes Congestion-Aware Traffic Engineering for LEO Mega-Constellations

Do Phuc Hao

***Abstract*—Low Earth orbit (LEO) mega-constellations carry traffic over a dense mesh of inter-satellite links whose topology, although time varying, is known in advance from orbital ephemeris. Under load, routing on propagation delay alone concentrates traffic on a few bottleneck links while alternative paths sit idle, wasting a large fraction of the achievable end-to-end performance. The congestion-aware optimum, the user equilibrium, removes this waste but requires the full demand matrix and an iterative solve at every slot, which does not scale to constellations of more than a thousand satellites with rapidly changing topology. We propose an inductive graph neural network that predicts the equilibrium congestion price on each inter-satellite link in a single forward pass, from cheap and observable load features, and routes every flow on the resulting cost. The model is trained on small shells and applied without retraining to larger ones. On a constellation simulator with a standard Bureau of Public Roads cost model, the network matches the user equilibrium in distribution while using one to two orders of magnitude less computation, recovers about four fifths of the gain when transferred zero shot to a shell twice as large, and, when fed the predicted near future demand, sustains low travel time under drifting traffic hotspots where a reactive one slot lagged controller degrades below even congestion-blind shortest path. We also report a negative result: a quantum-walk propagation operator, which motivated this study, offers no benefit once load features are available, and a plain graph convolution is preferable. We release the full pipeline and the honest record of the design search.**

***Index Terms*—LEO satellite networks, non-terrestrial networks, routing, traffic engineering, graph neural networks, congestion control, user equilibrium, inductive learning.**

I. INTRODUCTION

LOW Earth orbit (LEO) mega-constellations such as Starlink and Kuiper interconnect their satellites with laser inter-satellite links (ISLs) arranged in a regular grid, forming a wide-area packet network that moves with the constellation. Two features distinguish this network from a terrestrial one. First, the topology is time varying but *deterministic*: the position of every satellite, and therefore every ISL and its propagation delay, is known in advance from the orbital parameters. Second, the network is large and fast: a single shell contains on the order of a thousand or more satellites, and cross-plane links open and close as satellites traverse the constellation.

Routing such a network on propagation delay alone is attractive because the shortest path is cheap to compute and, in a lightly loaded regime, near optimal. Under load, however, delay-only shortest path is wasteful: it funnels many flows onto the same minimum-hop links, creating hotspots, while geometrically longer but idle paths are ignored. We measure this waste directly (Section VI-A) and find that congestion-aware routing cuts total travel time by half or more once the network is moderately loaded.

The congestion-aware optimum is classical. Routing every flow so that no flow can unilaterally lower its cost yields the *user equilibrium* (UE), computed by iterating shortest-path assignment against a load-dependent link cost. UE is the right target, but it is expensive: it needs the entire demand matrix and many shortest-path rounds per slot. At constellation scale, with topology changing every few seconds, recomputing UE from scratch each slot is impractical.

This tension, a cheap-but-blind heuristic versus an expensive-but-optimal solve, motivates our contribution. We learn to *amortize* the equilibrium. A graph neural network (GNN) observes cheap features (per-node traffic intensity and the load that one congestion-blind pass would induce) and predicts, in a single forward pass, the equilibrium congestion price on every ISL. Routing on the predicted cost recovers most of the congestion-aware gain at a fraction of the cost of solving UE. Because the model is a message-passing network whose parameters do not depend on the number of satellites, it is *inductive*: trained on a small shell, it applies unchanged to a larger one. And because the demand pattern drifts predictably (the dense-traffic band rotates under the constellation), feeding the predicted near future demand lets the router act *proactively*, before congestion forms.

Contributions.

- **C1 (amortization).** A one-shot GNN matches the user equilibrium in distribution (total travel time within two percent) while using 20 to 27$\times$ less computation than the iterative solve (Sections IV, VI).
- **C2 (inductive scale generalization).** Trained on small shells, the same model transferred zero shot to a shell twice as large is the best *deployable* policy, recovering about 79% of the blind-to-UE gain and beating every cheap baseline.
- **C3 (cost).** We quantify the inference saving against UE and show it is the operative reason to learn rather than solve.
- **C4 (proactive).** Under drifting hotspots, routing on predicted demand tracks the clairvoyant optimum, whereas

D. P. Hao is with the Center for Artificial Intelligence Research and Application (CAIRA), Da Nang Architecture University, Da Nang, Vietnam (e-mail: haodp@dau.edu.vn). Corresponding author: D. P. Hao.

a reactive one-slot-lagged controller degrades below congestion-blind shortest path as the drift grows.

- **Honest negative result.** The study began as a quantum-walk GNN. We report, with the full kill-gate record, why that line did not survive contact with the data, and why a plain graph convolution is the right operator here.

II. Related Work

LEO routing and topology. The deterministic, time-varying structure of LEO ISL networks has been studied for low-latency routing and topology design [1], [2]. Packet-level simulators such as Hypatia [3] and StarPerf [4] reconstruct the moving topology from constellation parameters. These works route mostly on delay; we focus on the loaded regime where congestion dominates.

GNNs for networking. Graph convolutions [5], attention [6], and inductive sampling [7] underpin learning on graphs. RouteNet and related models predict per-path delay and jitter for network modeling [8]; reinforcement learning with GNNs has been applied to routing optimization [9]. Our model is supervised and predicts a congestion-price field that is decoded by shortest path, which keeps inference to a single forward pass and makes inductivity straightforward.

Traffic engineering and equilibria. Load-dependent link cost and the user equilibrium are classical in transportation [10], [11]; learning to route to reduce congestion has been explored for terrestrial networks [12], [13]. We bring this to LEO, where the value lies in amortizing the per-slot solve across a fast, scale-varying topology and in exploiting the predictable demand drift.

Quantum-walk networks. Continuous-time quantum walks have been used as a graph propagation operator [14]. We evaluate one and report it as a negative ablation here.

III. System Model and Problem

Constellation and links. A Walker shell of S satellites with P planes is propagated on circular orbits. Each satellite holds two intra-plane ISLs and up to two cross-plane ISLs, forming a +Grid. An ISL (u, v) has a propagation delay prop_{uv} equal to its length over the speed of light, and a capacity cap. The topology over a horizon is a known sequence of graphs G_t.

Demand and cost. A demand set $\mathcal{D} = \{(s, d, r)\}$ requests rate r from s to d. We draw demands from a gravity model whose hotspots concentrate near a longitude band that rotates between slots, a proxy for the rotating dense-population/sub-solar region. The realized cost of a link under load x_{uv} follows the Bureau of Public Roads (BPR) form,

$$\mathrm{cost}_{uv}(x) = \mathrm{prop}_{uv}\Big(1 + \alpha\,(x_{uv}/\mathrm{cap})^{\beta}\Big), \quad \alpha = 0.15,\ \beta = 4. \tag{1}$$

This is smooth and monotone, so routing has a well defined user equilibrium (UE), the fixed point at which no flow can lower its cost by rerouting. We compute UE by the method of successive averages (MSA) over shortest-path assignments against (1).

References that bound the design. Two policies bound any router. Blind shortest path routes on prop alone: one assignment pass, congestion blind. UE routes on the equilibrium cost: congestion optimal, but it needs the full demand matrix and many MSA rounds per slot, which does not scale.

Problem. We seek a map f_θ from cheap, observable state to a routing that approaches UE in one forward pass and generalizes across shell size. We target the UE congestion price

$$g^*_{uv} = \mathrm{cost}^{\mathrm{UE}}_{uv}/\mathrm{prop}_{uv} - 1 \ \geq 0, \tag{2}$$

and route every flow on $\mathrm{prop}_{uv}\,(1 + \hat{g}_{uv})$ using the prediction $\hat{g}$.

IV. Method

Features. For a slot we form per-node features $[\,\text{out-demand, in-demand, blind out-load, blind in-load}\,]$ and per-edge features $[\,\text{prop, blind load}\,]$. The *blind load* is the load that one congestion-blind shortest-path pass would induce; it is cheap (a single assignment, no iteration) and is the single most informative input, since the equilibrium price on a link is largely a function of how overloaded that link would be under naive routing. All features are scaled to be size invariant.

Network. An edge-price GNN embeds node features through K message-passing layers and reads out a per-edge price

$$\hat{g}_{uv} = \mathrm{softplus}\big(\mathrm{MLP}([\,h_u,\, h_v,\, \mathrm{prop}_{uv},\, \mathrm{blindload}_{uv}\,])\big), \tag{3}$$

trained by mean-squared error to $\log(1+g^*)$ pooled over many (shell, demand) instances. The propagation is a symmetric-normalized graph convolution; this operator choice is justified empirically in Section VI-D. Because the parameters are independent of the satellite count, the trained model applies to any shell.

Decoding and proactivity. Given $\hat{g}$, routing is one shortest-path assignment on prop $(1+\hat{g})$. For the proactive variant, the features are formed from the *predicted* next-slot demand (here, the known-drift forecast), so the router pre-empts the moving hotspot rather than chasing it.

V. Experimental Setup

We simulate Walker shells with a pure-kinematic propagator (no packet-level dependency, fully reproducible from parameters): an Iridium-like shell (66 satellites, diameter 9) and Starlink-like 53° shells of 132 (d=16), 264 (d=28), and 1584 satellites. Capacity is uniform; demands follow the drifting gravity model. We report total travel time (TTT) relative to blind shortest path (lower is better; blind = 1.00), averaged over multiple demand instances and random seeds with standard deviations. Baselines: *blind*, *geographic* greedy (great-circle next hop, with a shortest-path fallback so all flows are delivered), *one-step* congestion correction (route on one blind-load pass), *GNN* (ours), and *UE* (the congestion-aware reference). Every learned result is param-matched across operators and reported with error bars; metrics are survivorship free (no silently dropped flows).

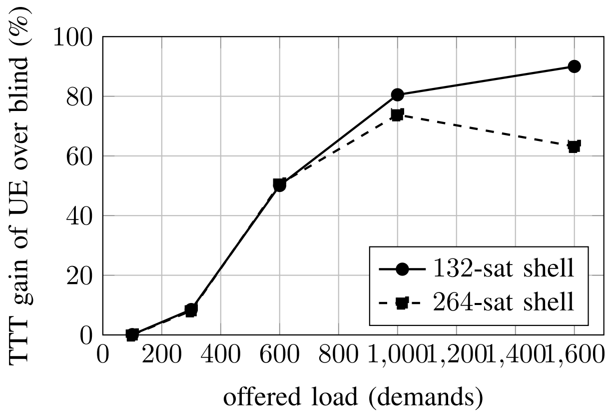

Fig. 1. Congestion headroom: the travel-time gain of the user equilibrium over congestion-blind shortest path grows with load. At light load there is no gain; under load, blind routing wastes half or more of the achievable performance.

TABLE I
TOTAL TRAVEL TIME RELATIVE TO BLIND SHORTEST PATH (LOWER IS BETTER; BLIND = 1.00). MEAN OVER INSTANCES AND SEEDS. UE IS THE EXPENSIVE REFERENCE; THE GNN IS THE DEPLOYABLE POLICY.

split	UE (ref.)	geographic	one-step	**GNN**	blind
in-distribution	0.46	0.78	1.04	**0.47**	1.00
OOD (2× shell)	0.23	0.80	0.55	**0.39**	1.00

VI. RESULTS

A. There is large headroom for congestion awareness

Figure 1 sweeps offered load. At light load, blind shortest path is already near optimal and UE offers nothing. As load grows, blind concentrates traffic on bottleneck links (its maximum link utilization reaches several times capacity while alternates idle) and UE recovers 8, 50, 80, and 90% of travel time. The congestion task, not static routing, is where a learned router can help.

B. The GNN matches UE cheaply and transfers (C1, C2, C3)

Table I reports TTT relative to blind. In distribution, the GNN (0.47) matches UE (0.46): it captures essentially all of the congestion-aware gain. Transferred zero shot to a shell twice as large, the GNN (0.39) is the best *deployable* policy, recovering about 79% of the blind-to-UE gain and beating every cheap baseline. The cheap alternatives fail: one-step correction overshoots and oscillates (worse than blind in distribution), and geographic greedy stalls on the ISL grid, requiring shortest-path fallback for 53 to 68% of flows. The GNN reaches this quality at 20 to 27× less computation than the MSA solve (Table II).

C. Proactive routing under drifting demand (C4)

Figure 2 sweeps the per-slot hotspot drift. At zero drift the reactive and proactive routers are identical, as they must be. As the hotspot moves, the reactive one-slot-lagged router routes for yesterday's load and degrades, passing blind shortest path at moderate drift and reaching 1.6× blind at the largest drift. The proactive router, fed the predicted demand, tracks the clairvoyant equilibrium throughout. The proactive advantage grows from 0 to 76%.

TABLE II
INFERENCE COST: ONE GNN FORWARD PASS PLUS ROUTING VERSUS THE MSA SOLVE FOR THE USER EQUILIBRIUM (WALL-CLOCK, CPU).

shell	sats	GNN (s)	UE/MSA (s)	speedup
132-sat	132	0.10	2.77	27×
264-sat	264	0.71	13.7	19×

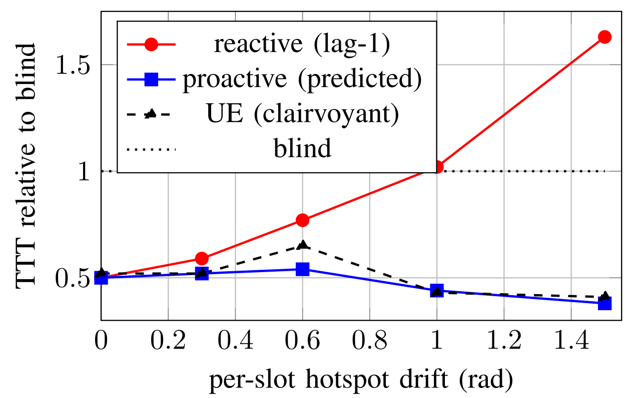

Fig. 2. Proactive versus reactive routing under drifting traffic hotspots. The reactive lag-1 router degrades past blind as the hotspot moves; the proactive router, using predicted demand, tracks the clairvoyant optimum.

D. Operator ablation, including the quantum walk (negative)

Table III compares three param-matched propagation operators: a local graph convolution (GCN), a global classical-diffusion operator (Heat), and a global quantum-walk operator (QW). Once the blind-load features are present, predicting the price is largely a local function of a link's own load, so the plain GCN is best and the global operators add variance without benefit. The quantum-walk operator, which originally motivated this work as a way to capture long-range structure, is *not* preferred. We keep it here as an honest negative ablation.

VII. DISCUSSION AND LIMITATIONS

The simulator is kinematic and flow level, not packet level; it captures geometry-driven delay and load-induced congestion through the BPR model, which is the mechanism the method targets, but it does not model queueing dynamics or protocol behavior. The user equilibrium is a selfish equilibrium and not the system optimum, which is why some policies record travel time slightly below UE; a system-optimal reference would tighten the upper bound. The zero-shot transfer leaves a residual gap to UE on the larger shell, driven partly by one high-variance seed; training across a range of shell sizes is a natural way to close it. Finally, the proactive variant assumes the demand drift is predictable, which holds for the diurnal rotation of the dense-traffic band but not for sudden, unforecastable surges.

TABLE III
OPERATOR ABLATION: RECOVERED FRACTION OF THE BLIND-TO-UE GAIN, ZERO SHOT TO THE LARGER SHELL (MEAN $\pm$ STD OVER INSTANCES AND SEEDS). THE PLAIN GCN IS BEST; THE QUANTUM-WALK OPERATOR DOES NOT HELP.

operator	in-distribution	OOD ($2\times$ shell)
GCN (local)	$99 \pm 9\%$	$\mathbf{88 \pm 17}\%$
Heat (global diffusion)	$103 \pm 9\%$	$70 \pm 13\%$
QW (global quantum walk)	$95 \pm 10\%$	$85 \pm 32\%$

VIII. CONCLUSION

We presented an inductive graph network that amortizes congestion-aware traffic engineering for LEO mega-constellations. It predicts the user-equilibrium congestion price on every inter-satellite link in one forward pass from cheap load features, matches the equilibrium in distribution at one to two orders of magnitude less computation, transfers zero shot to larger shells as the best deployable policy, and, fed predicted demand, sustains low travel time under drifting hotspots where reactive control fails. We also documented why an initially appealing quantum-walk operator did not earn its place, and released the full pipeline and design record so the negative results are reusable.

REFERENCES

[1] M. Handley, “Delay is not an option: Low latency routing in space,” in *Proc. ACM HotNets*, 2018.
[2] D. Bhattacherjee and A. C. Snoeren, “Network topology design at 27,000 km/hour,” in *Proc. ACM CoNEXT*, 2019.
[3] S. Kassing, D. Bhattacherjee, A. B. Águas, J. E. Saethre, and A. Singla, “Exploring the ‘Internet from space’ with Hypatia,” in *Proc. ACM IMC*, 2020.
[4] Z. Lai, H. Li, and J. Li, “StarPerf: Characterizing network performance for emerging mega-constellations,” in *Proc. IEEE ICNP*, 2020.
[5] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. ICLR*, 2017.
[6] P. Veličković *et al.*, “Graph attention networks,” in *Proc. ICLR*, 2018.
[7] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Proc. NeurIPS*, 2017.
[8] K. Rusek *et al.*, “RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN,” *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, 2020.
[9] P. Almasan *et al.*, “Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case,” *Computer Communications*, vol. 196, 2022.
[10] J. G. Wardrop, “Some theoretical aspects of road traffic research,” *Proc. Institution of Civil Engineers*, 1952.
[11] Bureau of Public Roads, *Traffic Assignment Manual*, U.S. Dept. of Commerce, 1964.
[12] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, “Learning to route,” in *Proc. ACM HotNets*, 2017.
[13] Z. Xu *et al.*, “Experience-driven networking: A deep reinforcement learning based approach,” in *Proc. IEEE INFOCOM*, 2018.
[14] S. Dernbach, A. Mohseni-Kabir, S. Pal, and D. Towsley, “Quantum walk neural networks,” in *Complex Networks*, 2018.